

Foundations of LLM agents, and the state of the art

How the models work, why they work, and what you build around them.

Josh Speagle

Astronomy & Astrophysics · Statistical Sciences, University of Toronto



David A. Dunlap Department of Astronomy & Astrophysics
UNIVERSITY OF TORONTO

JOSH SPEAGLE . COM

Created with assistance from Claude

Built with Claude.
Every idea is mine.
None of the slides are.

The process that built this talk is the worked example in part 3.

Four things to take away

How they work

A model turns a context into the next token.
An agent is that model with tools.

Part 1 · 10 min

Why they work

Predicting varied text, without memorizing,
forces general strategies.

Part 2 · 8 min

The harness

Everything around the model is scaffolding
you build and control.

Part 3 · 13 min

Working with them

Clear vision, clear communication, clear
execution. Design first.

Part 4 · 8 min

Where we are

How they work

A model turns a context into the next token.
An agent is that model with tools.

Part 1 · 10 min

Why they work

Predicting varied text, without memorizing,
forces general strategies.

Part 2 · 8 min

The harness

Everything around the model is scaffolding
you build and control.

Part 3 · 13 min

Working with them

Clear vision, clear communication, clear
execution. Design first.

Part 4 · 8 min

Text becomes tokens

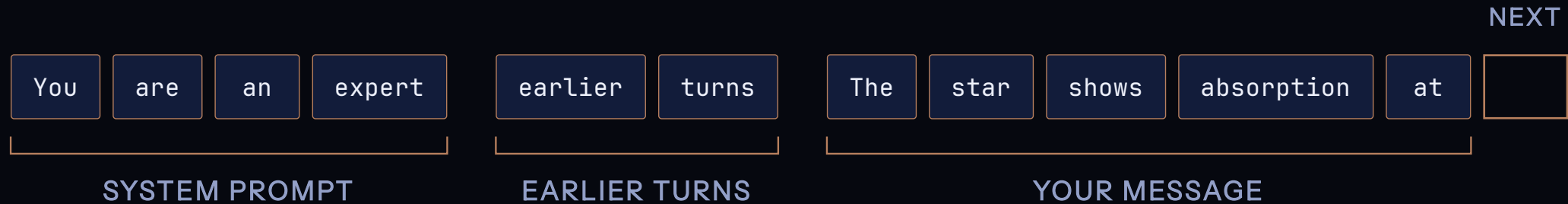
SUB-WORD PIECES, AND THEIR INDICES

Ast	roph	ysics	_is	_a	_statistical	_science
62152	22761	17688	374	264	29564	8198

A token is a piece of text with a fixed index in a vocabulary of about 100,000 entries. A word can be one token or several. The model never sees letters or words, only indices.

Already know this? Tokenization and the chat template are covered properly in John Wu's SSAIL tutorial, linked on the last slide.

The context is everything the model sees



Everything the model knows about this problem is in this strip. Nothing else is visible to it, and nothing is remembered between runs unless it is put back in.

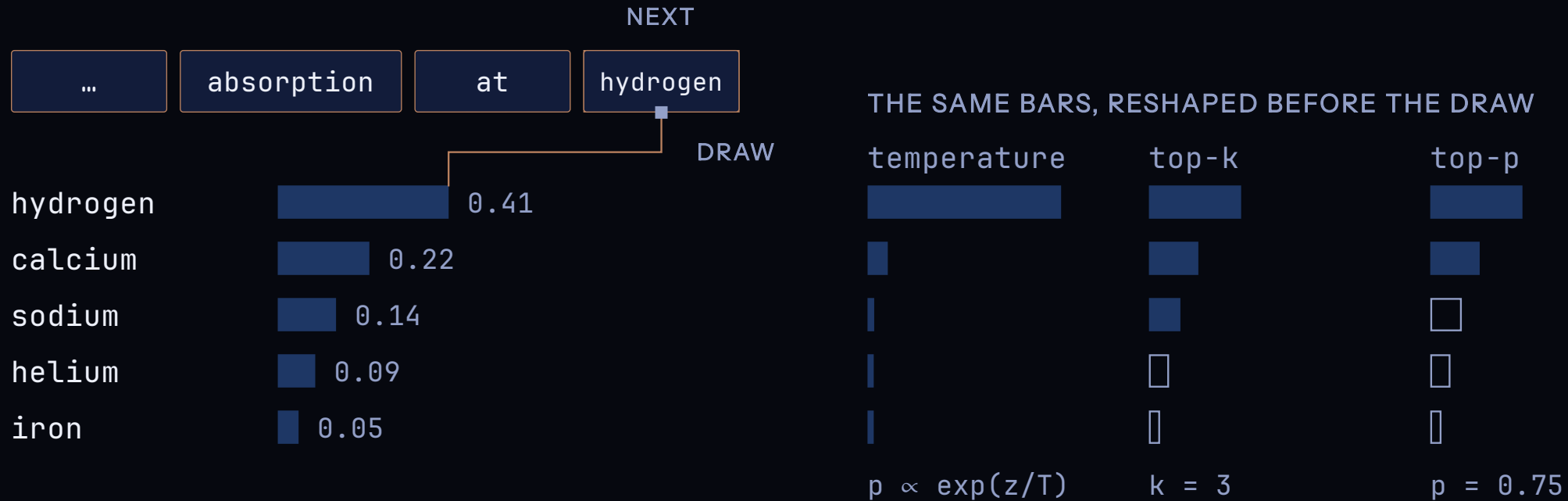
One pass, one distribution



One forward pass gives one probability for every token in the vocabulary. The bars are the whole output.

Values illustrative.

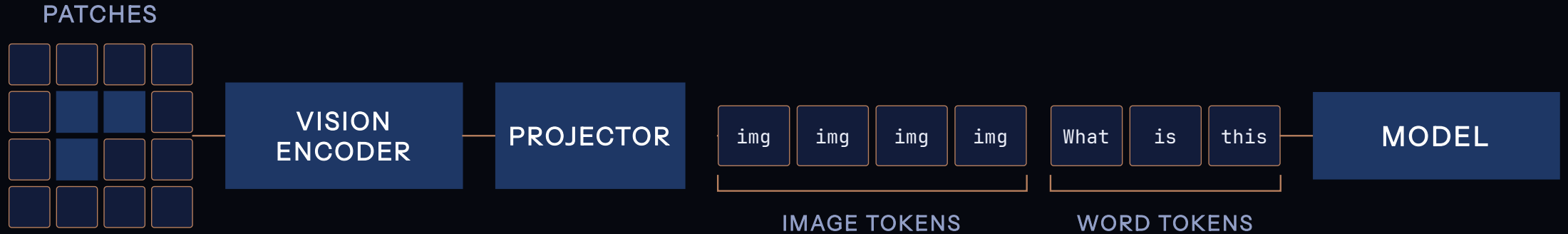
Choosing the next token is a random draw



The next token is drawn at random from the distribution. That draw is why the same prompt gives different answers. Temperature, top-k and top-p only reshape the bars before the draw; none of them checks anything.

Even at temperature 0, servers are not bit-for-bit repeatable: the result depends on what else is in the batch (Thinking Machines, 2025).

Images become tokens too



An image is cut into patches. A vision encoder turns each patch into a vector, a learned projector maps those vectors into the model's token-embedding space, and they are inserted into the sequence where the image sits. From there the model treats them like any other tokens.

LLaVA-1.5: 576 tokens per image · Qwen2-VL: 256-1024, set by resolution

How a model refers to a tool

Tools described in the prompt

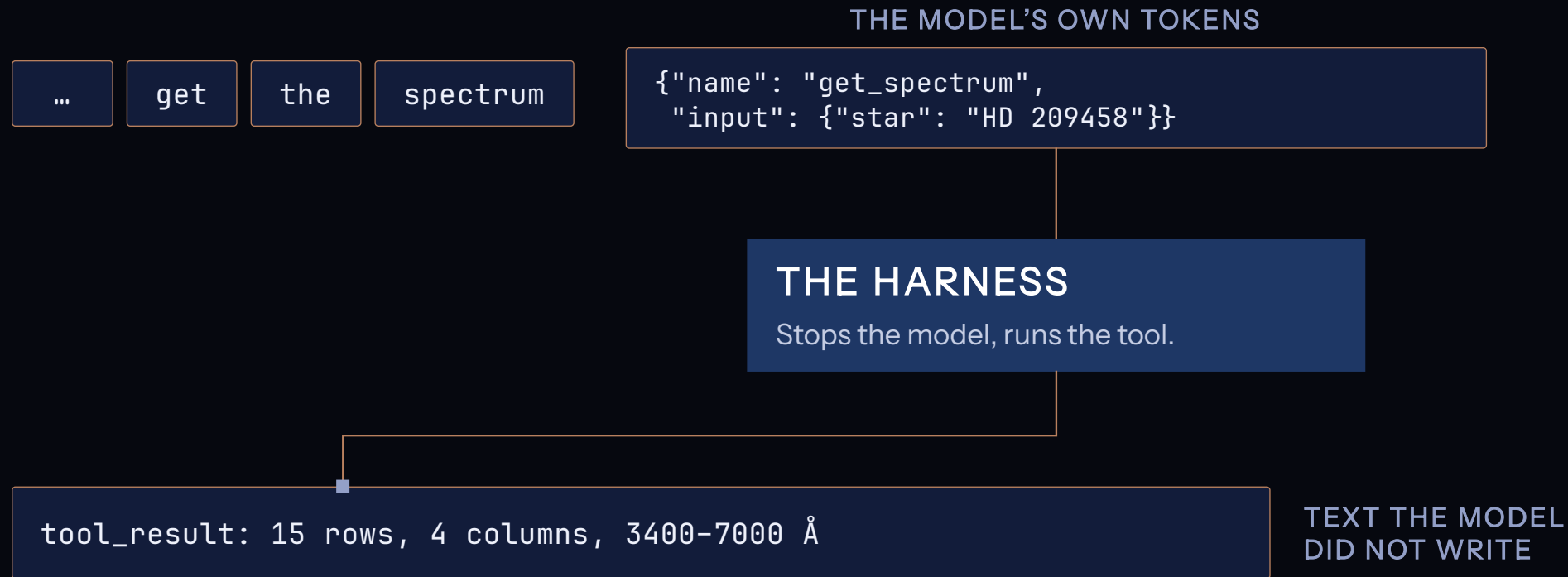
The harness puts each tool's name, description and argument schema into the system prompt. The model answers with structured text, `{"name": ..., "input": {...}}`, and the harness parses it. This is how the Anthropic and OpenAI APIs work.

Tools as special tokens

Open models have dedicated vocabulary entries for this: Llama's `<python_tag> ... </python_tag>`, Qwen's `<tool_call> ... </tool_call>`, Mistral's `[TOOL_CALLS]`. End-of-turn tokens and reasoning delimiters like `<think>` are the same kind of thing.

Either way, the model learned in post-training to emit these in the right place. The tool itself is code the harness runs.

When the model calls a tool



The harness stops the model at the tool call, runs the tool, and appends the result to the context. The model then continues from a context that now contains text it did not produce.

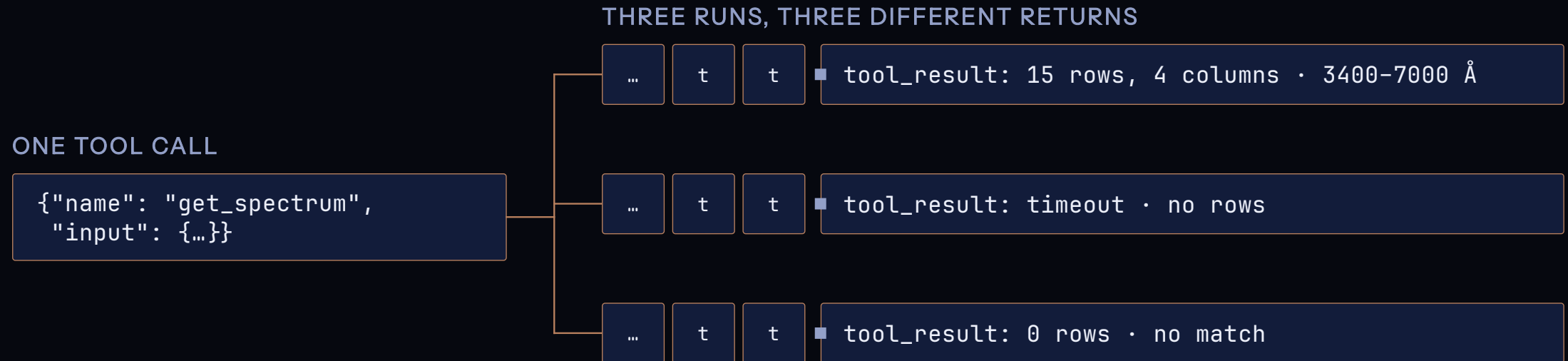
The same exchange as the API records it

```
▪  
{  
  "role": "assistant",  
  "content": [  
    {  
      "type": "tool_use",  
      "id": "toolu_01A09q90qw90lq917835lq9",  
      "name": "get_weather",  
      "input": {  
        "location": "New York, NY",  
        "unit": "fahrenheit"}}]]  
  
    {  
      "role": "user",  
      "content": [  
        {  
          "type": "tool_result",  
          "tool_use_id": "toolu_01A09q90qw90lq917835lq9",  
          "content": "15 degrees Celsius,  
                    partly cloudy"}}]]  
    }
```

- 1 The model emitted structured tokens and stopped.
- 2 The result comes back as a user message. The tool speaks in the other party's turn.
- 3 The model never predicted these tokens. In agentic RL training they are masked out of the loss (Wang & Ammanabrolu 2025).

Source · Anthropic tool-use docs · Wang & Ammanabrolu 2025

The result is not what the model expected



Chain of thought is the model's own text: it is written and read by the same process. A tool result is different. It is produced elsewhere, it varies from call to call, and it can contradict what the model assumed. The model has to read it rather than predict it, and errors in it get acted on.

A model predicts the next token from its context.

An agent is that model with tools, and every result comes back into the context.

Nothing else is added. Everything else is built around this.

Where we are

How they work

A model turns a context into the next token.
An agent is that model with tools.

Part 1 · 10 min

Why they work

Predicting varied text, without memorizing,
forces general strategies.

Part 2 · 8 min

The harness

Everything around the model is scaffolding
you build and control.

Part 3 · 13 min

Working with them

Clear vision, clear communication, clear
execution. Design first.

Part 4 · 8 min

The training task

MASKED · BERT, 2018

The [] of the galaxy

distance		0.62
radius		0.19
mass		0.11

NEXT TOKEN · GPT-2, 2019

The distance of the []

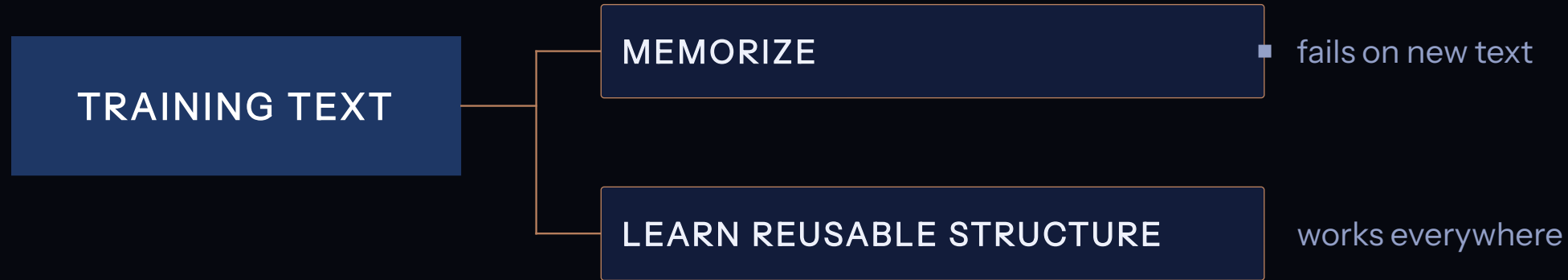
galaxy		0.55
cluster		0.17
nebula		0.08

REUSABLE STRUCTURE

Predict the missing token, or the next one, across trillions of tokens of text that is different every time. Masking (BERT, 2018) and next-token prediction (GPT-2, 2019) both produced representations that transferred to tasks they were never trained on.

Source · Devlin 2018, arXiv:1810.04805 · Radford 2019 · values illustrative

No memorizing, so it learns strategies



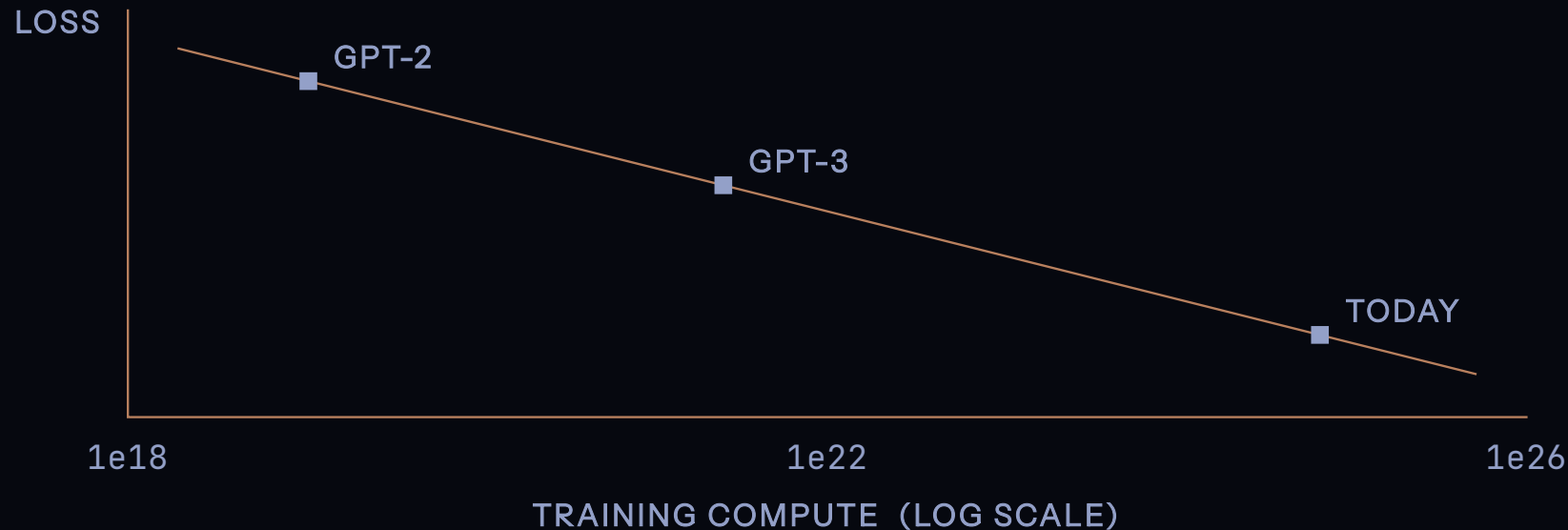
The text is new at every step, parts of it are hidden, and dropout removes parts of the network. Anything that only works on examples the model has seen is punished. Anything that works everywhere is kept. If the only thing that works everywhere is an algorithm, the network learns the algorithm. Small transformers trained on regression examples learn to run gradient descent inside a forward pass (von Oswald 2022).

Generating tokens lets it run the algorithm



One forward pass has a fixed depth. Generating a token and reading it back adds a step, and there is no limit on steps. That is what a reasoning trace is: the model running a procedure long enough to finish it.

Scale shows the objective keeps working

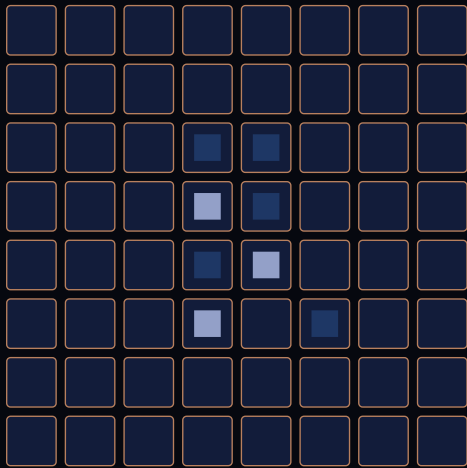


Loss falls as a power law over seven orders of magnitude of compute, and the details of the architecture matter far less than scale (Kaplan 2020; Hoffmann 2022). Attention and pre-training are the Oct 5 talk.

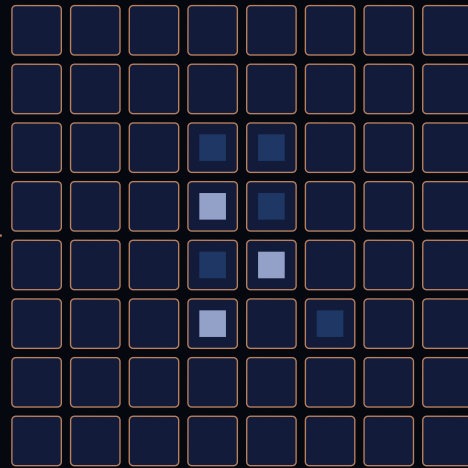
Source · Kaplan 2020, arXiv:2001.08361 · Hoffmann 2022, arXiv:2203.15556 · positions schematic

Representations are the by-product

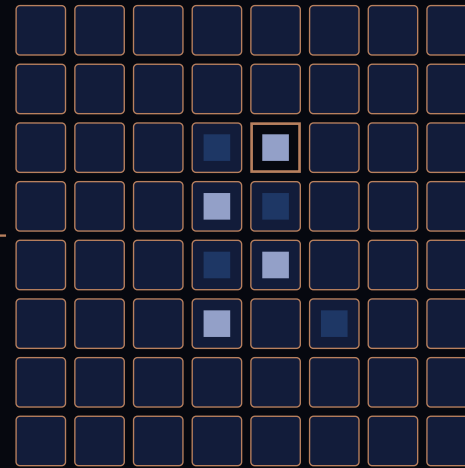
THE BOARD (NEVER SHOWN)



PROBE READOUT



AFTER INTERVENTION



ONE CELL EDITED

A model trained only on Othello move sequences, never shown a board, keeps a linear representation of the board, and editing that representation changes its moves (Li 2023; Nanda 2023). Llama-2 activations linearly encode the latitude, longitude and dates of real places and events (Gurnee & Tegmark 2023). The cheapest way to predict text about the world is to represent the world. Interpretability is the Dec 7 talk.

The same principle at every later stage

PRE-TRAINING

Representations.

FINE-TUNING

Format, instructions.

RL WITH FEEDBACK OR VERIFIED ANSWERS

Strategies that get
checked.

RL ON TOOLS

Strategies that transfer
across tools.

Few-shot learning appears because examples-then-answer is a pattern in text (Brown 2020). RL against verifiable answers produces self-checking and long reasoning without being shown them (DeepSeek-R1, 2025). RL on tool use in one domain transfers to domains never trained on (RITE 2025). Each stage is the same bet: a hard, varied objective and no way to memorize. Finetuning and RL are the Oct 19 talk.

A model that only predicts text is not an agent.

Post-training taught it to follow instructions, to reason, and to call tools.

Now it needs somewhere to act.

That is the harness.

Where we are

How they work

A model turns a context into the next token.
An agent is that model with tools.

Part 1 · 10 min

Why they work

Predicting varied text, without memorizing,
forces general strategies.

Part 2 · 8 min

The harness

Everything around the model is scaffolding
you build and control.

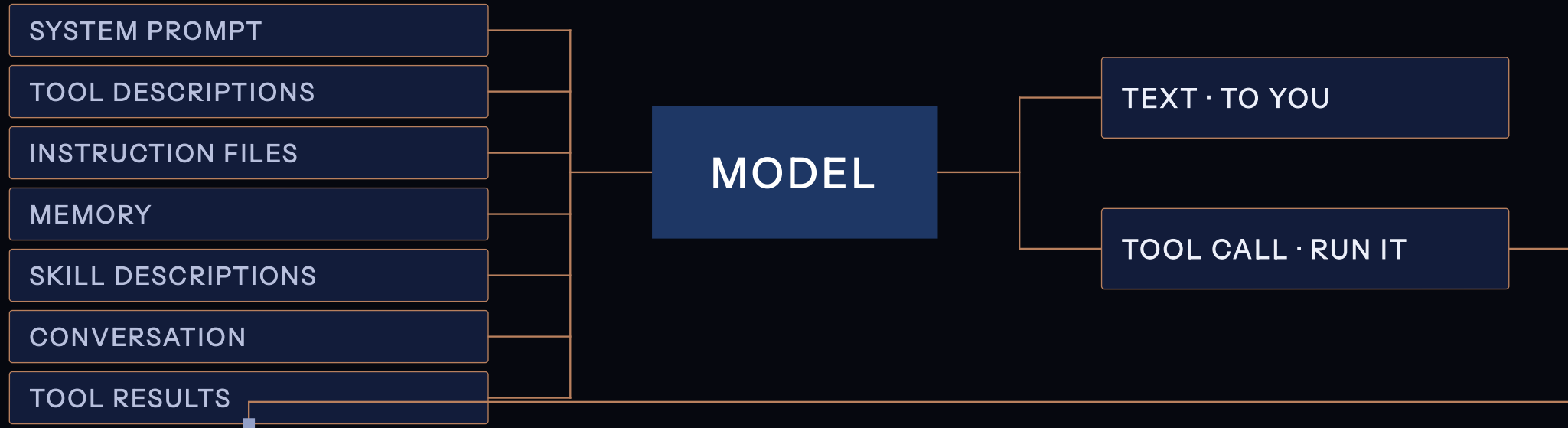
Part 3 · 13 min

Working with them

Clear vision, clear communication, clear
execution. Design first.

Part 4 · 8 min

The harness assembles the prompt



Before you type anything, the harness has already filled most of the context. It decides what goes in, in what order, which tools exist, and who approves an action. The model is the same; the harness is where the design lives.

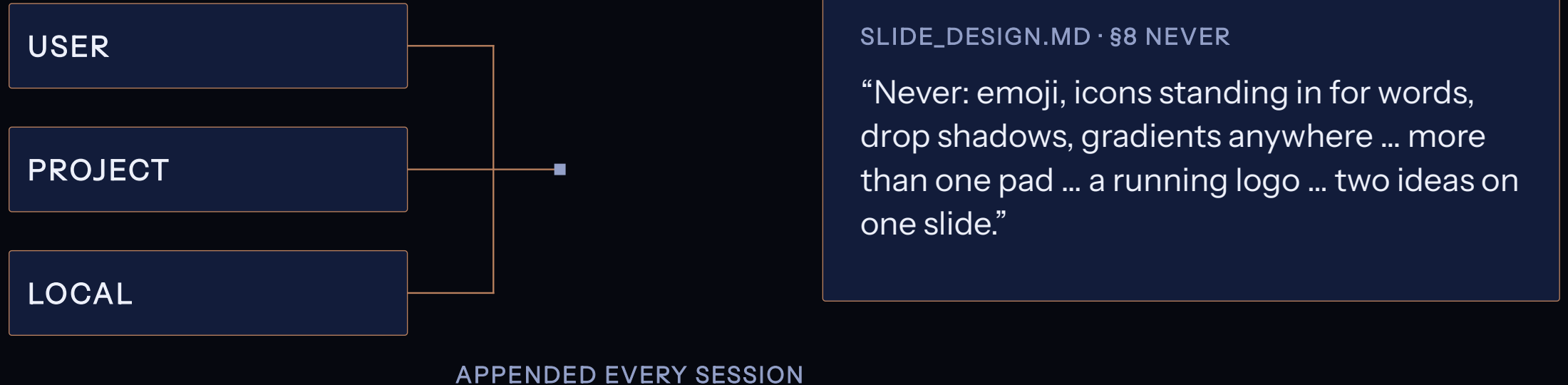
Three harnesses, the same models

	CLAUDE CODE	CODEX CLI	PI
INSTRUCTION FILES	CLAUDE.md, four levels, merged	AGENTS.md chain, 32 KB cap	13-line prompt + AGENTS.md
TOOLS	~40 built-in + MCP	one shell tool + patch	read, write, edit, bash
APPROVALS	six modes + rules	approval × sandbox	none
MEMORY	auto-memory files	opt-in flag	no
SUBAGENTS	yes, own context	opt-in flag	no; re-run via bash
SANDBOX	built-in + container	seatbelt / bwrap	none; use Docker

Claude Code sets every axis. Codex gives one shell tool inside an approval-by-sandbox matrix. pi is under a thousand tokens, four tools and no permission prompts. Same models, very different agents.

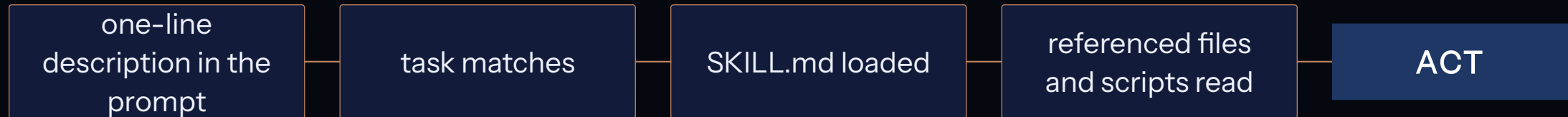
Source · code.claude.com · learn.chatgpt.com · mariozechner.at (Nov 2025)

Instruction files



An instruction file is text the harness appends at the start of every session: your standards, your rules, your specs. The spec that built these slides is one. It is 17 KB and it has outlived every conversation that used it.

Skills load when they are needed



THE FOLDER THAT BUILT THIS DECK

```
_design-system/design/SLIDE_DESIGN.md  
_design-system/slides/lib/speagle_slides.py  
_design-system/slides/lib/README.md
```

A skill is a folder of instructions, scripts and files. Only its one-line description sits in the prompt. When a task matches, the harness loads the file, and the file tells the model which other files to read. The slide system is a skill; the builder read the spec, then the library's API, then the plan.

Memory is written, then read back later

WRITTEN, THEN READ

SESSION 1

MEMORY FILE

SESSION 2'S PROMPT

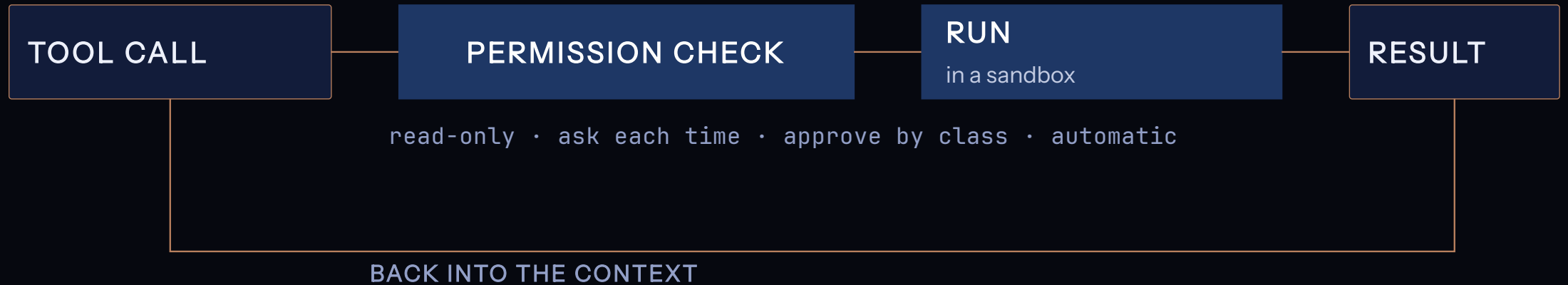
[stated] Talks folder reorganized Sep 2026: year folders, YYYY-MM-DD_venue_topic file names, TALKS.md index regenerated by _index/build_index.py

[stated] Slide design preference: integrated visual + textual representation on many slides; figures must reinforce and explain the point, never an image for its own sake

[stated] Opening talk of the NASA Cosmic Origins AI/ML STIG fall 2026 lecture series (Mon Sep 14 2026, 4pm ET, Teams), ~50 min overview meant to set up the follow-up talks

Nothing persists between runs unless it is written to a file and put back into the prompt. Memory files are how the same preferences reached every agent that worked on this talk. Retrieval and memory are the Oct 26 talk.

Tools and permissions



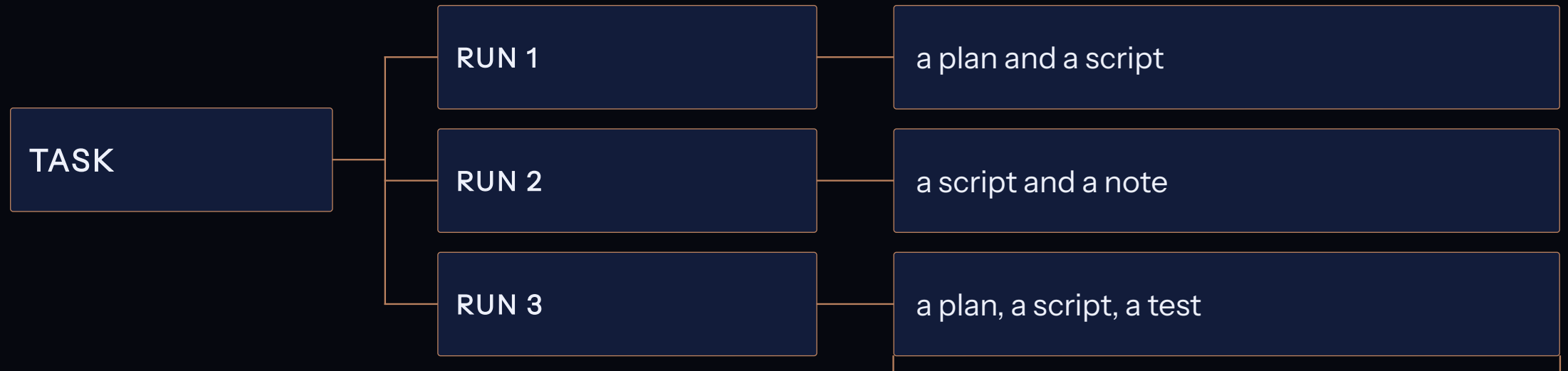
Every call passes a check the harness owns. The modes run from asking about everything to asking about nothing. MCP is an open standard for plugging external tools into any harness. Tool use and MCP are the Nov 2 talk.

Agents as tools



A subagent is the same model with a different context, so it behaves differently. It works in its own window and returns only a summary, which keeps the lead's context small and lets work run in parallel. Cost: agents use about four times the tokens of a chat, multi-agent systems about fifteen (Anthropic 2025). Orchestration is the Nov 16 talk.

The same task, run three times



SIMILAR, NOT IDENTICAL

Outputs are drawn, not computed. Three runs of the same delegation land in similar places, not the same place, and you cannot reproduce a run exactly. Design for it: ask for outputs you can check, put the requirements in writing, and add a verification step.

The harness that built this talk



A · web · 2,472 words · 81 tool uses · 7.2 min

B · own decks · 1,900 words · 35 tool uses · 6.8 min

C · why they work · 1,484 words · 16 tool uses · 2.1 min

builder v2 · 46 slides · 107 tool uses · 24.5 min

Three research agents, one builder, one lead, one human. The rule that cheaper models implement and the lead plans was written down before anything ran. Two tool results said no tonight (a blocked shell, a locked log file); both were read and worked around, which is what the check step is for.

Same model, different harness, different agent.
The harness is the part you control.

Prompt assembly, tools, permissions, memory, skills, subagents:
each is a decision.

Where we are

How they work

A model turns a context into the next token.
An agent is that model with tools.

Part 1 · 10 min

Why they work

Predicting varied text, without memorizing,
forces general strategies.

Part 2 · 8 min

The harness

Everything around the model is scaffolding
you build and control.

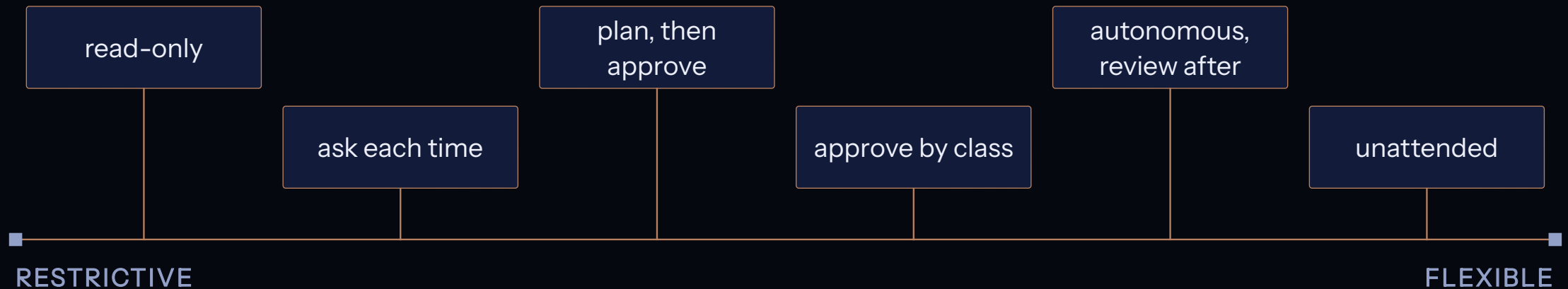
Part 3 · 13 min

Working with them

Clear vision, clear communication, clear
execution. Design first.

Part 4 · 8 min

Restrictive to flexible



Every harness sits somewhere on this axis, and one workflow can move along it: restrictive while you set the task up, flexible once it is well specified, restrictive again to check. Where you sit should change as the models change and as your work changes.

Three ways to work with a model



The further right, the less you see and the more the specification has to carry. Autonomy concentrates the audit.

Vision, communication, execution

CLEAR VISION

Know what you want.

CLEAR COMMUNICATION

Say it so someone could act on it without you.

CLEAR EXECUTION

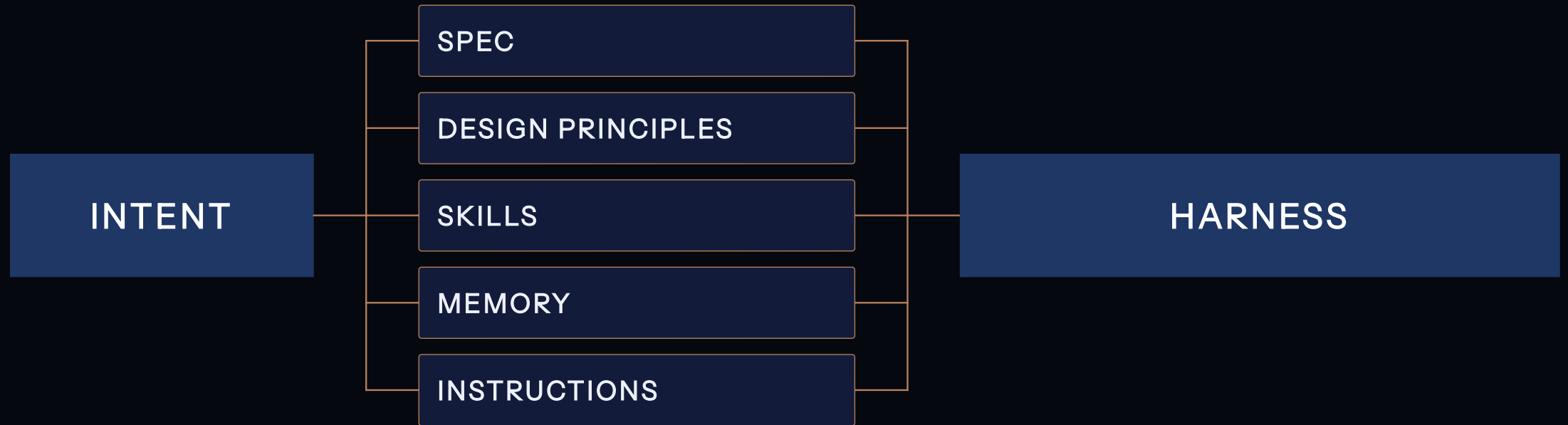
Then it executes.

YOURS

THE AGENT'S

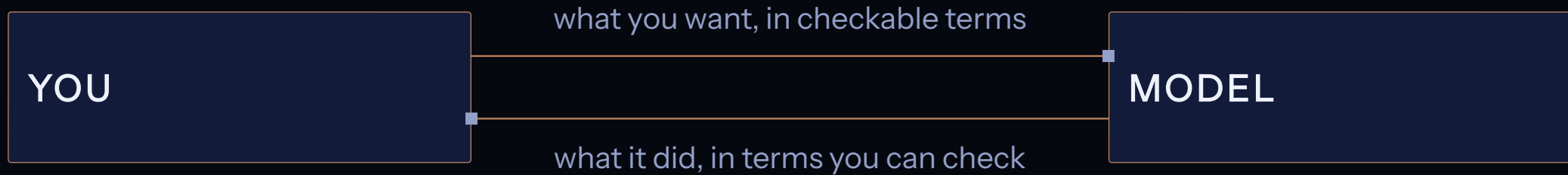
Know what you want. Say it so that an agent, or a colleague, could act on it without you. Then it executes. The first two are your work, and they are most of the work. Do them before the model starts, not after.

Turn intent into things the model reads



Anything you want every run to respect goes into a file the harness loads. Written once, read every time. Keep the files short and current; prune what stops being true. This is backwards design: decide the outcome, then build what produces it.

Communication runs both ways



You need the model to report in a form you can verify: assumptions stated, sources resolved, changes listed. That is a skill you can write. Do not accept output you cannot read.

What it costs when you skip this

19% slower, 20% faster

Experienced developers using early-2025 AI tools were 19% slower while believing they were 20% faster (METR 2025).

50 against 67

People who shipped code with AI assistance scored 50% on comprehension of it, against 67% without (Anthropic, Jan 2026); the gap closed for those who asked conceptual questions.

Treated like ordinary code, they cost time

These systems over-engineer by default and run for hours in the wrong direction if the target is vague. Treated like ordinary code, they cost time. Treated as systems you design and monitor, they return it.

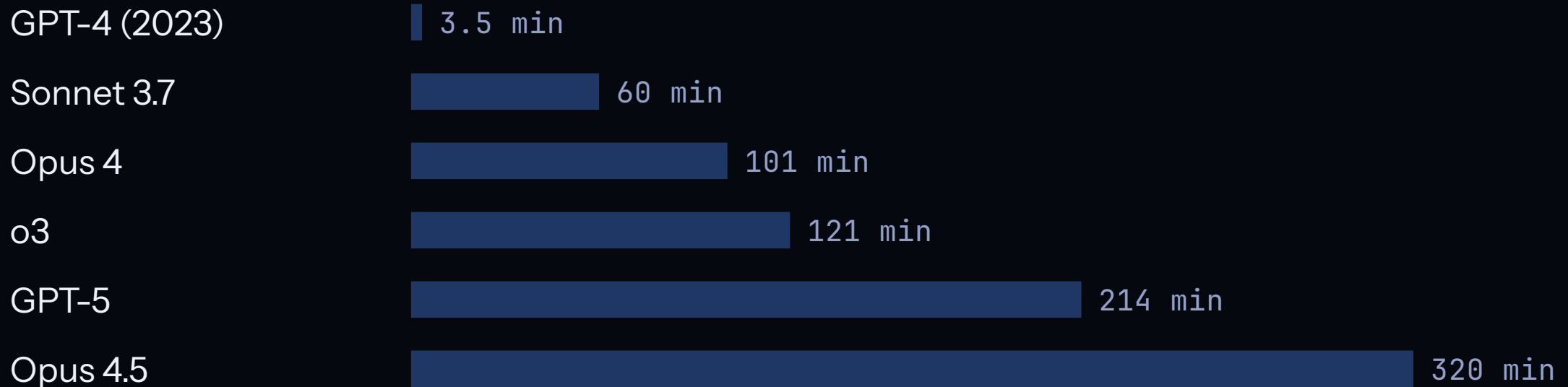
Source · METR, Jul 2025 and Feb 2026 · Anthropic, Jan 2026

Be the showrunner, not the audience.

John Wu. Specifying the goal is the work.

How long a task can an agent finish

50% SUCCESS TIME HORIZON, IN HUMAN WORKING TIME



METR measures the length, in human working time, of software tasks that an agent completes half the time. It has doubled about every seven months since 2023, about every three since 2024. Caveats: the tasks are almost all coding, and the top bar's interval runs from about two to about twenty hours.

Source · METR Time Horizon 1.1, Jan 2026

What agents did for this talk

TASK

Reorganize a 3.7 GB folder of 270 files

Index 176 talks with tags

Research: web, own decks, mechanism

Build 46 slides with 22 figures

Write the plan and every line of text

Decide scope, structure, what to drop

WHO · HOW LONG

script, run by the human

script

three agents, 7, 7 and 2 min

one agent, 25 min, three render
passes

lead

human, four checkpoints

The scope has moved from lines of code to whole tasks. The human's work moved to the start (the brief, the decisions) and the end (the review).

In our field

ReplicationBench

ReplicationBench (2025) asked agents to reproduce results from 20 astrophysics papers, 111 tasks. The best frontier agent managed 22%; failures were giving up early, missing domain knowledge and old data formats.

Kosmos

Kosmos, an AI-scientist system: 79% of its statements were judged accurate, 58% of its synthesis statements.

Retrieval for cosmology

cmbagent's retrieval setup answered 91% of cosmology questions correctly.

Where to read more

AstroMLab, AstroReview, and the STIG textbook (Part V: LLM basics, RAG, MCP, agents).

Source · ReplicationBench, [arXiv:2510.24591](#) · Kosmos, [arXiv:2511.02824](#) · cmbagent, [arXiv:2507.07155](#)

Four things to take away

How they work

Context in, a distribution out, one draw appended; a tool result is text the model did not write.

Why they work

No memorizing, varied data, a hard objective: strategies, and representations as the by-product.

The harness

Prompt assembly, tools, permissions, memory, skills, subagents. You choose where it sits between restrictive and flexible.

Working with them

Vision, communication, execution. Design first. Intent into files. Monitor the harness.

Where to go from here

Next in the series

Sep 21 and 28: hands-on agentic coding
(Stubbs, Saad)
ai4astro.org

John Wu's SSAIL tutorial

Tokens, context, prompting
8 Jun 2026

Slides, worksheet, design system

joshspeagle.com/talks
14 Sep 2026

Contact

j.speagle@utoronto.ca
joshspeagle.com

